目录

基于 CNN 的校园课堂智能考勤系统 — 测试方案文档 **1**
1. 测试目标 . . . 1
2. 单元测试方案 . . . 2
   2.1 测试框架 . . . 2
   2.2 AI 模块单元测试 . . . 2
   2.3 业务服务层单元测试 . . . 4
3. 集成测试方案 . . . 5
   3.1 测试环境 . . . 5
   3.2 认证接口测试 . . . 5
   3.3 考勤核心接口测试 . . . 6
   3.4 人脸数据接口测试 . . . 6
4. AI 模型评估方案 . . . 6
   4.1 评估数据集划分 . . . 6
   4.2 评估指标 . . . 7
   4.3 VGG16 vs ResNet50 对比实验 . . . 7
   4.4 混淆矩阵分析 . . . 7
5. 性能测试方案 . . . 7
   5.1 接口响应时间测试 . . . 7
   5.2 AI 推理性能测试 . . . 8
6. 安全测试方案 . . . 9
7. 测试执行计划 . . . 9

基于 CNN 的校园课堂智能考勤系统 — 测试方案文档

版本: v1.0
日期: 2026-04-16
依据: 需求分析文档 v1.1 / 详细设计文档 v1.0

---

1. 测试目标

测试类型	目标
单元测试	验证各模块核心函数的逻辑正确性，覆盖率 70%
集成测试	验证 API 接口的端到端正确性，覆盖全部 P0 接口
AI 模型评估	验证 VGG16 与 ResNet50 在人脸识别任务上的性能指标

测试类型	目标
性能测试	验证系统满足响应时间和并发要求
安全测试	验证认证、权限、输入校验等安全机制

---

2. 单元测试方案

2.1 测试框架

- **框架**: pytest
- **Mock 工具**: pytest-mock / unittest.mock
- **覆盖率工具**: pytest-cov
- **运行命令**: `pytest tests/unit/ --cov=app --cov-report=html`

2.2 AI 模块单元测试

TC-U-01：人脸检测器正常检测

```python
# tests/unit/test_face_detector.py
def test_detect_single_face(face_detector, sample_face_image):
    """ 正常人脸图片应检测到 1 个人脸"""
    bboxes = face_detector.detect(sample_face_image)
    assert len(bboxes) == 1
    assert bboxes[0].confidence >= 0.9
    assert bboxes[0].x2 > bboxes[0].x1
    assert bboxes[0].y2 > bboxes[0].y1

def test_detect_no_face(face_detector, blank_image):
    """ 纯色图片应返回空列表"""
    bboxes = face_detector.detect(blank_image)
    assert bboxes == []

def test_detect_multiple_faces(face_detector, group_photo):
    """ 多人合影应检测到多个人脸"""
    bboxes = face_detector.detect(group_photo)
    assert len(bboxes) >= 2
```

TC-U-02：特征数据库匹配逻辑

```python
# tests/unit/test_feature_database.py
def test_query_above_high_threshold(feature_db, registered_student_vec):
    """ 相似向量应返回 present 状态"""
    result = feature_db.query(registered_student_vec)
    assert result['status'] == 'present'
    assert result['confidence'] >= 0.75

def test_query_in_middle_zone(feature_db, noisy_student_vec):
    """ 加噪声后的向量应返回 unverified 状态"""
    result = feature_db.query(noisy_student_vec)
    assert result['status'] == 'unverified'
    assert 0.5 <= result['confidence'] < 0.75

def test_query_unknown_person(feature_db, random_vec):
    """ 随机向量应返回 unknown 状态"""
    result = feature_db.query(random_vec)
    assert result['status'] == 'unknown'
    assert result['confidence'] < 0.5

def test_query_empty_database():
    """ 空特征库应返回 unknown"""
    db = FeatureDatabase()
    result = db.query(np.random.randn(512))
    assert result['status'] == 'unknown'
```

TC-U-03：预处理器

```python
def test_preprocess_output_shape(preprocessor, raw_face_image):
    """ 预处理后张量形状应为 (1, 3, 224, 224)"""
    tensor = preprocessor.preprocess(raw_face_image, mock_bbox)
    assert tensor.shape == (3, 224, 224)

def test_preprocess_normalized_range(preprocessor, raw_face_image):
    """ 归一化后像素值应在合理范围"""
    tensor = preprocessor.preprocess(raw_face_image, mock_bbox)
    # ImageNet 归一化后大致在 [-2.5, 2.5]
    assert tensor.min() >= -3.0
    assert tensor.max() <= 3.0
```

2.3 业务服务层单元测试

TC-U-04：考勤记录去重

```python
# tests/unit/test_attendance_service.py
def test_duplicate_recognition_ignored(attendance_service, mock_db):
    """ 同一学生在同一任务中第二次识别应被忽略"""
    attendance_service.record_if_new(task_id=1, student_id='2022001', confidence=0.9)
    attendance_service.record_if_new(task_id=1, student_id='2022001', confidence=0.88)
    assert mock_db.session.add.call_count == 1

def test_different_tasks_not_deduplicated(attendance_service, mock_db):
    """ 不同任务的相同学生不去重"""
    attendance_service.record_if_new(task_id=1, student_id='2022001', confidence=0.9)
    attendance_service.record_if_new(task_id=2, student_id='2022001', confidence=0.9)
    assert mock_db.session.add.call_count == 2
```

TC-U-05：Excel 导入校验

```python
def test_import_valid_excel(student_service, valid_excel_file):
    result = student_service.import_from_excel(valid_excel_file)
    assert result['success'] == 10
    assert result['failed'] == 0

def test_import_duplicate_student_id(student_service, duplicate_excel_file):
    """ 重复学号行应跳过并返回错误信息"""
    result = student_service.import_from_excel(duplicate_excel_file)
    assert result['failed'] > 0
    assert '学号已存在' in str(result['errors'])

def test_import_missing_required_field(student_service, incomplete_excel):
    result = student_service.import_from_excel(incomplete_excel)
    assert result['failed'] > 0
    assert '姓名字段为空' in str(result['errors'])
```

---

3. 集成测试方案

3.1 测试环境

- **测试数据库**：独立 MySQL 测试库（`attendance_test`），每次测试前回滚
- **框架**：pytest + Flask test client
- **Fixture**：`conftest.py` 中定义测试用 App、DB、JWT Token

```python
# tests/conftest.py
@pytest.fixture(scope='session')
def app():
    app = create_app('testing')
    with app.app_context():
        db.create_all()
        seed_test_data()
        yield app
        db.drop_all()

@pytest.fixture
def client(app):
    return app.test_client()

@pytest.fixture
def admin_token(client):
    resp = client.post('/api/auth/login',
                       json={'username': 'test_admin', 'password': 'Test@123'})
    return resp.json['data']['token']
```

3.2 认证接口测试

测试编号	用例描述	请求	预期状态码	预期结果
TC-I-01	正确账号密码登录	POST /api/auth/login	200	返回含 token 的 data
TC-I-02	错误密码登录	POST /api/auth/login	401	message 含提示信息
TC-I-03	禁用账号登录	POST /api/auth/login	403	message: 账号已禁用
TC-I-04	空密码登录	POST /api/auth/login	400	参数校验失败
TC-I-05	无 Token 访问受保护接口	GET /api/students	401	—
TC-I-06	过期 Token 访问	GET /api/students	401	—
TC-I-07	无权限角色访问	DELETE /api/students/1（教师角色）	403	—

3.3 考勤核心接口测试

测试编号	用例描述	请求	预期状态码
TC-I-10	创建考勤任务	POST /api/attendance/tasks	201
TC-I-11	开始考勤任务	POST /api/attendance/tasks/1/start	200
TC-I-12	重复开始同一任务	POST /api/attendance/tasks/1/start	409
TC-I-13	结束考勤任务	POST /api/attendance/tasks/1/stop	200
TC-I-14	结束后缺勤自动标记	GET /api/attendance/records?task_id=1	200
TC-I-15	人工修正记录	PUT /api/attendance/records/1	200
TC-I-16	学生角色修正他人记录	PUT /api/attendance/records/1（学生 Token）	403

3.4 人脸数据接口测试

测试编号	用例描述	预期状态码
TC-I-20	上传含有效人脸的图片	200，face_count +1
TC-I-21	上传无人脸的图片	422，“未检测到人脸”
TC-I-22	上传超大图片（>5MB）	413
TC-I-23	上传非图片文件	400
TC-I-24	上传至未注册学生	404

---

4. AI 模型评估方案

4.1 评估数据集划分

```
自建校园数据集（假设50人，每人10张）：
    训练集：每人7张（70%）
    验证集：每人2张（20%）
    测试集：每人1张（10%，不参与任何训练决策）

评估时额外测试：
    不同光照条件的图片
    不同角度（侧脸）图片
    戴眼镜/口罩的图片（鲁棒性测试）
```

4.2 评估指标

指标	公式	目标值
Top-1 准确率	正确识别数 / 总测试数	85%
精确率（Precision）	TP / (TP + FP)	0.85
召回率（Recall）	TP / (TP + FN)	0.85
F1 Score	2 × P × R / (P + R)	0.85
AUC-ROC	ROC 曲线下面积	0.90
单帧推理时间	ms/帧（含检测 + 提取 + 匹配）	200ms（CPU）
误识率（FAR）	FP / (FP + TN)	5%
拒识率（FRR）	FN / (FN + TP)	10%

4.3 VGG16 vs ResNet50 对比实验

实验维度	VGG16	ResNet50	说明
参数量	138M	25M	ResNet50 更轻量
特征维度	512d	512d（avgpool 后）	统一维度对比
训练时长（30 epoch）	待测	待测	GPU 下记录
Top-1 准确率	待测	待测	在测试集上
CPU 推理延迟	待测	待测	ms/帧
内存占用（推理）	待测	待测	MB

结论预期：ResNet50 在准确率相近的情况下，推理速度更快、参数更少，适合部署为默认模型。

4.4 混淆矩阵分析

对测试集每个学生生成混淆矩阵，重点检查：

- **高混淆对**（误识率最高的两人）：分析其特征向量的余弦相似度
- **低召回学生**：通常因注册图片质量差（光线不足、遮挡）

---

5. 性能测试方案

5.1 接口响应时间测试

工具：ab (Apache Bench) 或 locust

```bash
# 测试登录接口在 100 并发下的响应时间
ab -n 1000 -c 100 -p login.json -T application/json http://localhost:5000/api/auth/login

# 测试考勤记录查询（带分页）
ab -n 500 -c 20 -H "Authorization: Bearer <token>" \
   "http://localhost:5000/api/attendance/records?page=1&per_page=20"
```

目标:

接口	P50 响应时间	P99 响应时间
登录	200ms	500ms
学生列表查询	300ms	800ms
考勤记录导出（1000 条）	3s	10s

5.2 AI 推理性能测试

```python
# tests/performance/test_inference_speed.py
import time

def test_single_frame_inference_time():
    """ 单帧推理时间应 <= 200ms（CPU 环境）"""
    model_manager = ModelManager.get_instance()
    test_image = load_test_image()

    times = []
    for _ in range(100):  # 100 次取均值
        start = time.perf_counter()
        results = model_manager.recognize_faces(test_image)
        elapsed = (time.perf_counter() - start) * 1000
        times.append(elapsed)

    avg_time = sum(times) / len(times)
    p99_time = sorted(times)[98]

    assert avg_time <= 200, f" 平均推理时间 {avg_time:.1f}ms 超出目标"
    assert p99_time <= 500, f"P99 推理时间 {p99_time:.1f}ms 超出目标"
```

6. 安全测试方案

测试项	测试方法	通过标准
SQL 注入防护	在查询参数中注入 `' OR 1=1--`	返回 400，不执行注入
JWT 伪造	使用错误密钥签名的 Token 访问接口	返回 401
越权访问	学生 Token 访问管理员接口	返回 403
文件上传类型绕过	上传重命名为 `.jpg` 的 PHP 文件	返回 400，内容类型校验失败
密码暴力破解防护	连续 10 次错误登录	（P2）触发限流或账号锁定
敏感信息泄露	检查 500 错误响应是否包含堆栈信息	生产模式下不暴露 traceback

---

7. 测试执行计划

阶段	时间	内容
单元测试	开发阶段同步	每个模块完成后立即编写单元测试
集成测试	后端 API 完成后	全接口回归测试
AI 模型评估	训练完成后	在测试集上运行评估脚本，生成对比报告
性能测试	系统集成完成后	使用 ab/locust 压测
安全测试	上线前	手动 + 工具扫描

文档版本：*v1.0* | 最后更新：*2026-04-16*